

# ProteinDreamer: Latent Diffusion World Models for Multi-Step Protein Engineering

PhD Research Proposal

Loqman Samani

Email: [samani10qman91@gmail.com](mailto:samani10qman91@gmail.com)

Webpage: <https://loqmansamani.github.io>

March 2026

# Contents

|          |                                                                             |           |
|----------|-----------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                         | <b>5</b>  |
| <b>2</b> | <b>Literature Review</b>                                                    | <b>6</b>  |
| 2.1      | Protein Representation, Fitness Prediction, and Generative Design . . . . . | 6         |
| 2.2      | Reinforcement Learning and Active Learning for Protein Engineering . . .    | 7         |
| 2.3      | World Models, JEPA, and Active Inference . . . . .                          | 7         |
| <b>3</b> | <b>Research Questions and Hypotheses</b>                                    | <b>8</b>  |
| 3.1      | Research Questions . . . . .                                                | 8         |
| 3.2      | Hypotheses . . . . .                                                        | 9         |
| <b>4</b> | <b>Methodology</b>                                                          | <b>10</b> |
| 4.1      | Problem Formulation . . . . .                                               | 10        |
| 4.2      | System Architecture Overview . . . . .                                      | 10        |
| 4.3      | Protein Encoder . . . . .                                                   | 10        |
| 4.4      | World Model: Latent Diffusion JEPA (Architecture A) . . . . .               | 11        |
| 4.5      | Alternative: Energy-Based JEPA (Architecture B) . . . . .                   | 12        |
| 4.6      | Reward Head . . . . .                                                       | 12        |
| 4.7      | Active Inference Policy . . . . .                                           | 12        |
| 4.8      | Training Procedure . . . . .                                                | 13        |
| 4.9      | Data . . . . .                                                              | 13        |
| <b>5</b> | <b>Expected Contributions and Timeline</b>                                  | <b>13</b> |
| 5.1      | Scientific Contributions . . . . .                                          | 13        |
| 5.2      | Timeline . . . . .                                                          | 14        |
| <b>6</b> | <b>Starting Project: JEPA World Models for Protein Fitness Landscapes</b>   | <b>14</b> |
| <b>A</b> | <b>Mathematical Details</b>                                                 | <b>20</b> |
| A.1      | Notation . . . . .                                                          | 20        |
| A.2      | MDP Formulation . . . . .                                                   | 20        |
| A.3      | Context Encoder . . . . .                                                   | 21        |
| A.4      | Fusion MLP . . . . .                                                        | 21        |
| A.5      | Action Encoder . . . . .                                                    | 21        |
| A.6      | Target Encoder (EMA) . . . . .                                              | 21        |
| A.7      | Architecture A: Latent Diffusion JEPA . . . . .                             | 21        |
| A.8      | Architecture B: Energy-Based JEPA . . . . .                                 | 22        |
| A.9      | Reward Head . . . . .                                                       | 22        |
| A.10     | Uncertainty Quantification . . . . .                                        | 22        |

|                                            |           |
|--------------------------------------------|-----------|
| A.11 Active Inference Policy . . . . .     | 23        |
| <b>B Training and Inference Algorithms</b> | <b>23</b> |

## Abstract

Protein engineering today relies on two dominant strategies that each leave something on the table. Generative models such as ProteinMPNN, RFdiffusion, and ESM-3 produce candidate sequences in a single forward pass, with no mechanism for iterative refinement or multi-step planning. Directed evolution, on the other hand, performs sequential search through mutation and screening, but does so blindly and at great experimental cost. Neither approach does what a thoughtful engineer would do: build an internal model of how mutations reshape the fitness landscape, then plan a sequence of edits by mentally simulating their consequences before committing to the wet lab.

This proposal introduces **ProteinDreamer**, a model-based reinforcement learning framework that learns a latent-space world model of the protein fitness landscape and uses it to plan multi-step mutation trajectories entirely in imagination. The system encodes protein states (sequence and predicted structure) into a compact latent representation using ESM-2 and GVP-GNN, then predicts post-mutation latent states through a conditional denoising diffusion model operating in that same low-dimensional space. A reward head translates latent states into fitness predictions (stability, binding affinity, catalytic activity), and an Active Inference policy selects mutations by minimizing expected free energy, which naturally balances reward seeking with uncertainty reduction. Because all planning occurs in latent space, ProteinDreamer can dream thousands of mutation trajectories efficiently without invoking expensive structure predictors or wet-lab assays at each step.

The primary world model architecture, Latent Diffusion JEPA, is new: it combines the intervention-aware prediction of Causal-JEPA with the distributional accuracy of diffusion-based dynamics, moved from observation space into JEPA latent space. A simpler Energy-Based JEPA alternative provides a fast, deterministic baseline. The system will be evaluated on ProteinGym deep mutational scanning benchmarks and mega-scale stability data, with the long-term goal of interfacing with active learning loops for real protein engineering campaigns.

# 1 Introduction

Proteins are the molecular machines of life. Engineering them to carry out new functions, catalyze industrial reactions, or bind therapeutic targets with high affinity is one of the central ambitions of modern biotechnology. The sequence space is enormous: for a protein of length  $L = 300$ , there are  $20^{300}$  possible amino acid sequences. The fitness landscape over this space, the mapping from sequence to function, is rugged, epistatic, and only partially observed [1]. These properties make brute-force search impractical and demand intelligent navigation strategies.

Two broad computational paradigms have emerged. *Generative protein design* uses deep learning to produce candidate sequences in a single forward pass. ProteinMPNN [2] designs sequences conditioned on a fixed backbone, RFdiffusion [3] generates novel backbones through 3D denoising diffusion, and ESM-3 [4] provides a multimodal foundation model that jointly reasons over sequence, structure, and function. These methods are powerful, but they are fundamentally one-shot: they generate a candidate and hope it works, with no mechanism for iterative refinement or look-ahead planning over sequences of mutations. *Directed evolution*, the experimental counterpart, is sequential by nature (mutate, screen, select, repeat) yet current computational tools do not model it as a sequential decision-making problem [5, 6].

A parallel development in artificial intelligence has produced a class of agents that do exactly this kind of sequential planning: *world model* agents. Systems like DreamerV4 [7] learn an internal simulator of their environment and train policies by imagining future trajectories, achieving strong performance across diverse domains with remarkable sample efficiency. More recently, the Joint-Embedding Predictive Architecture (JEPA) paradigm [8], advanced by Causal-JEPA [9] and LeWorldModel [10], has shown that world models can operate entirely in latent space without reconstructing observations, making them faster and more scalable. DIAMOND [11] demonstrated that replacing the dynamics model with a diffusion process yields more accurate long-horizon rollouts and captures multi-modal transitions better than deterministic alternatives.

Despite these advances, the world model paradigm has not been applied to protein engineering. The closest existing work, DynaPPO [12], uses a simple MLP surrogate to approximate the fitness function and trains a policy on it, but lacks a structured world model that predicts full state transitions, incorporates structural information, or balances exploration with exploitation in a principled manner. More recent methods like  $\mu$ Protein [13] combine fitness predictors with RL-based search, and EvoPlay [14] applies self-play tree search, but none of these build an internal model of how the protein changes after a mutation, and none of them plan in a learned latent space.

This proposal presents **ProteinDreamer**, a framework that fills this gap. The key idea is to frame protein design as a Markov Decision Process where an agent proposes

mutations (actions), observes changes in a learned latent representation of the protein (state transitions), receives fitness predictions (rewards), and plans entire mutation trajectories in imagination before committing to evaluation. The world model is a Latent Diffusion JEPA, a new architecture that marries the intervention-conditioned prediction of Causal-JEPA with diffusion-based dynamics operating in latent space. The policy is grounded in Active Inference, minimizing expected free energy to obtain a principled balance between exploiting high-fitness regions and exploring uncertain parts of the landscape [15, 16].

## 2 Literature Review

### 2.1 Protein Representation, Fitness Prediction, and Generative Design

Protein language models (PLMs) have become the foundation for computational protein engineering. ESM-2 [17], a 650M-parameter Transformer trained on 65 million sequences, extracts evolutionary and structural information whose attention maps converge to physical residue-residue contacts and whose masked marginals provide competitive zero-shot fitness estimates. ESMFold, built on top of ESM-2, predicts 3D structures  $60\times$  faster than AlphaFold2 [18], enabling rapid evaluation of designed candidates. On the structural side, GVP-GNN [19] processes scalar and vector features on protein backbone atoms with rotational equivariance, capturing packing density, secondary structure, and geometric constraints in a lightweight architecture. S3F [20] demonstrated that integrating ESM-2 sequence embeddings, GVP-GNN structural features, and surface topology achieves state-of-the-art fitness prediction with only 20M trainable parameters, underscoring the value of multi-scale representations.

For benchmarking and training, ProteinGym [21] standardizes evaluation across 217 substitution and 66 indel deep mutational scanning (DMS) assays covering over 2.7 million variants, while the mega-scale dataset of Tsuboyama et al. [22] provides over 776,000  $\Delta G$  measurements across 479 domains, making it the richest available resource for learning stability landscapes. Chen et al. [23] showed that heterogeneous DMS data from multiple proteins can be integrated via shared biophysical representations for cross-landscape transfer.

The dominant paradigm in generative protein design is one-shot generation. Protein-MPNN [2] assigns sequences to fixed backbones with high experimental success. RFDiffusion [3] generates novel backbones via denoising diffusion on 3D coordinates. ESM-3 [4], a 98-billion-parameter multimodal model, jointly models sequence, structure, and function. EvoDiff [24] applies diffusion in discrete sequence space, and ProGen2 [25] uses autoregressive language modeling for family-specific generation. These methods produce candidates in one pass with no mechanism for iterative planning or multi-step optimization.

ProteinDreamer fills this gap with an explicit world model that predicts how mutations reshape the landscape and a policy that plans trajectories toward desired properties.

## 2.2 Reinforcement Learning and Active Learning for Protein Engineering

DynaPPO [12] is the foundational work applying model-based RL to biological sequence design, training an MLP surrogate and using it as a simulator for PPO optimization with uncertainty-based safeguards against model exploitation. EvoPlay [14] introduced Monte Carlo tree search with self-play for directed evolution, but uses a discriminative fitness oracle rather than a generative world model.  $\mu$ Protein [13] combines  $\mu$ Former (a pairwise masked language model with multi-scale scoring) and  $\mu$ Search (PPO with Dirichlet exploration noise), representing the current state of the art in RL-guided protein engineering. However, all these approaches use the fitness predictor as a black-box reward with no learned dynamics and no latent-space planning.

Lee et al. [26] proposed protein sequence design via model-based RL in a VAE latent space, the most directly related prior work. ProteinDreamer can be understood as the full-scale realization of that concept, replacing the simple VAE with pre-trained PLM encoders, adding world model dynamics and multi-step planning, and grounding exploration in Active Inference. Renard et al. [27] applied AlphaZero-style MCTS to protein backbone design, while ProteinRL [28] and Subramanian et al. [29] compared RL algorithms for sequence design.

GFlowNets [30] offer a diversity-seeking alternative, sampling candidates with probability proportional to reward to ensure broad landscape coverage. Within ProteinDreamer’s modular framework, GFlowNets represent an alternative policy class for applications prioritizing diversity. The adaptive machine learning paradigm for protein engineering [31] formalizes the iterative design loop of train-propose-test-retrain, which ProteinDreamer instantiates with a full world model replacing simple surrogates, and Active Inference replacing acquisition functions. Recent work on co-optimizing fitness and diversity [32] confirms that exploration-exploitation balance improves design outcomes, a property that expected free energy minimization achieves by construction.

## 2.3 World Models, JEPA, and Active Inference

The idea of learning an environment model for planning traces back to Dyna [33] and was revived by Ha and Schmidhuber [34]. A limitation of reconstruction-based approaches is that they waste capacity on observation details irrelevant for decision-making. The JEPA paradigm [8], developed into concrete world models by LeWorldModel [10], addresses this by discarding the decoder entirely, predicting future latent states conditioned on actions with SIGReg regularization, achieving planning speeds up to  $48\times$  faster. Causal-JEPA [9]

extended this with object-level latent interventions, learning to predict how interventions on individual entities change the world state, which is directly analogous to mutations as causal interventions on protein states.

On the dynamics modeling side, DIAMOND [11] replaced RSSM dynamics with a diffusion model, producing more accurate long-horizon rollouts and better capturing multi-modal transitions. DIAMOND operates in observation space (pixels), however, which is computationally expensive. Moving diffusion from observation space to latent space yields massive efficiency gains, the same insight behind Latent Diffusion Models (LDMs) [35]. This motivates ProteinDreamer’s Latent Diffusion JEPA, synthesizing DIAMOND’s distributional dynamics with JEPA’s latent-space efficiency.

The policy side is grounded in Active Inference [15], a framework from computational neuroscience where agents select actions to minimize expected free energy  $G(\pi)$ , decomposing into pragmatic value (reward seeking) and epistemic value (uncertainty reduction). Tschantz et al. [16] bridged active inference and deep RL through the Free Energy of the Expected Future, showing that deep ensembles for uncertainty estimation achieve strong model-based RL performance. Sajid et al. [36] formalized the distinction from standard RL, where active inference treats exploration as a first-principle requirement rather than an engineering heuristic. The connection to JEPA is deep, since both minimize variational free energy, and the JEPA energy scoring prediction consistency is mathematically analogous to the prediction error in variational free energy. This unification means ProteinDreamer’s world model, policy, and exploration strategy all derive from a single variational objective.

## 3 Research Questions and Hypotheses

### 3.1 Research Questions

1. **Can a JEPA-based world model learn an accurate, generalizable simulator of protein fitness landscapes from deep mutational scanning data?** Current fitness predictors map sequences directly to scalar fitness values. A world model would go further, predicting full latent state transitions (encoding structure and function changes) after mutations. The question is whether the additional structure of state-transition prediction provides better multi-step planning than flat fitness regression.
2. **Does a latent diffusion dynamics model outperform a deterministic predictor for capturing multi-modal, epistatic fitness landscapes?** Protein fitness landscapes are rugged. A single mutation can lead to distinct structural or functional outcomes depending on background context. A diffusion-based predictor generates distributional samples rather than a point estimate. The question is whether this

distributional capacity translates into better long-horizon rollouts and more effective exploration.

3. **Does Active Inference provide a principled and empirically superior exploration-exploitation strategy compared to standard RL for protein design?** The expected free energy objective automatically balances reward seeking with uncertainty reduction. The question is whether this principled mechanism outperforms heuristic exploration strategies (entropy bonuses, Dirichlet noise,  $\epsilon$ -greedy) on real protein fitness landscapes where data is scarce and every query is expensive.
4. **Can imagination-based planning in latent space discover multi-step mutation trajectories that cross fitness valleys unreachable by greedy single-step methods?** The signature advantage of world model agents is the ability to plan ahead. In protein engineering, this means crossing fitness valleys, sequences of mutations where each intermediate step is deleterious but the endpoint is beneficial. The question is whether ProteinDreamer can discover such trajectories in practice.

## 3.2 Hypotheses

- H1.** A world model trained with the JEPA paradigm (prediction loss + SIGReg) on DMS data will produce latent representations that predict fitness with accuracy competitive with state-of-the-art methods (ESM-2 zero-shot, S3F,  $\mu$ Former) and additionally enable accurate multi-step rollout planning, as measured by trajectory fitness correlation on held-out evolutionary paths.
- H2.** The Latent Diffusion JEPA (Architecture A) will produce more accurate multi-step rollouts than the Energy-Based JEPA (Architecture B) on rugged fitness landscapes, as measured by reduced prediction error compounding over 5–10 mutation steps and improved diversity among top-ranked candidate sequences.
- H3.** The Active Inference policy will discover higher-fitness sequences with fewer oracle queries than PPO with entropy bonus and  $\mu$ Search with Dirichlet noise, particularly in low-data regimes (<100 labeled variants), because the expected free energy objective directs exploration toward the most informative regions of sequence space.
- H4.** ProteinDreamer’s multi-step planning will identify mutation trajectories reaching fitness peaks inaccessible to greedy single-step optimization, validated on known evolutionary paths (e.g., TEM-1  $\beta$ -lactamase resistance evolution) where intermediate mutations are individually deleterious.

## 4 Methodology

### 4.1 Problem Formulation

We formulate protein design as a Markov Decision Process  $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$ . The state  $s_t$  consists of the current amino acid sequence  $\mathbf{x}_t \in \{A, C, D, \dots, Y\}^L$ , its predicted 3D structure  $\mathbf{C}_t \in \mathbb{R}^{L \times 3}$  (C $\alpha$  coordinates from ESMFold), and associated property estimates (pLDDT confidence, contact maps). The action  $a_t$  is a mutation, a single-site substitution  $(i, \text{AA}_{\text{new}})$  where  $i$  is the sequence position and  $\text{AA}_{\text{new}}$  is the replacement amino acid. The action space is  $\mathcal{A} = \{(i, j) : i \in \{1, \dots, L\}, j \in \{1, \dots, 20\}\}$ . Multi-mutant designs are achieved by chaining multiple single-step substitutions over a trajectory of horizon  $H$ . The current formulation does not include insertions or deletions (indels), which would require a variable-length action space and a world model capable of handling changing sequence lengths. Extending the framework to indels is an important long-term direction discussed in Section 5. The transition function  $\mathcal{T}$  is the world model, learned from data. The reward  $\mathcal{R}(z_t)$  is predicted fitness (stability  $\Delta\Delta G$ , binding affinity  $K_d$ , catalytic activity  $k_{\text{cat}}$ , or combinations). The discount factor  $\gamma = 0.99$  enables multi-step trajectory planning.

### 4.2 System Architecture Overview

The system consists of five components: (1) a protein encoder that maps raw protein states to latent representations, (2) a world model predictor that forecasts post-mutation latent states, (3) a reward head that predicts fitness from latent states, (4) an Active Inference policy that selects mutations, and (5) an uncertainty module that guides exploration. During planning, all computation occurs in latent space. No decoder or structure predictor is invoked. The policy dreams mutation trajectories of horizon  $H$  steps, evaluates them via the reward head, and commits to the best trajectory for evaluation.

### 4.3 Protein Encoder

The context encoder  $f_\theta$  maps a protein state  $s_t$  to a latent vector  $z_t \in \mathbb{R}^d$  ( $d \approx 256$ –512). It combines two complementary streams. A pre-trained ESM-2 [17] (650M parameters) processes the amino acid sequence  $\mathbf{x}_t$  to produce per-residue embeddings  $\mathbf{H}^{\text{seq}} \in \mathbb{R}^{L \times 1280}$ , which are mean-pooled to a global vector  $\mathbf{h}^{\text{seq}} \in \mathbb{R}^{1280}$ . A trainable GVP-GNN [19] processes the predicted 3D structure  $\mathbf{C}_t$  (here ESMFold will be used to predict 3D structure from sequence), represented as a graph where nodes are residues (with scalar and vector features from backbone geometry) and edges connect residues within a distance cutoff. Multiple message-passing layers produce per-residue structural embeddings, which are mean-pooled to a global vector  $\mathbf{h}^{\text{struct}}$ . The two global vectors are concatenated and projected through a fusion MLP to the final latent state  $z_t$  (see Appendix A for the full

encoder, Fusion MLP, and GVP-GNN equations).

A critical regularization is applied to  $z_t$ : SIGReg [10], which enforces  $z_t \sim \mathcal{N}(0, \mathbf{I})$  via random projection normality testing. SIGReg projects the batch of embeddings onto random unit-norm directions, tests univariate normality on each projection via the Epps-Pulley test, and aggregates via the Cramér-Wold theorem. It replaces VICReg’s six to seven hyperparameters with a single scalar  $\lambda_{\text{reg}}$ , and its Gaussian constraint provides a well-matched geometry for the diffusion process. Mutations are encoded into continuous action embeddings  $a_t^{\text{emb}}$  via a small MLP that combines positional encoding with amino acid identity embeddings.

A target encoder  $f_{\bar{\theta}}$ , an exponential moving average (EMA) copy of the context encoder with decay rate  $\tau_{\text{ema}}$  close to 1, provides stable prediction targets during training and prevents representational collapse.

#### 4.4 World Model: Latent Diffusion JEPA (Architecture A)

The primary world model predictor is a conditional denoising diffusion model [37] operating in the JEPA latent space, following the key insight of Latent Diffusion Models [35] that moving the diffusion process from observation space to a learned latent space yields large efficiency gains. Given the current latent state  $z_t$  and mutation embedding  $a_t^{\text{emb}}$ , it generates samples from  $p_{\theta}(z_{t+1}|z_t, a_t)$  via iterative denoising.

During training, the target latent  $\bar{z}_{t+1}$  (from the EMA encoder, stop-gradient) is noised at a random level  $\tau \sim \mathcal{U}(1, K)$  according to a cosine noise schedule following the DDPM formulation [37]. A small Transformer-based denoiser predicts the clean latent from the noised version, conditioned on  $z_t$  and  $a_t^{\text{emb}}$  via cross-attention. The training loss is a denoising score-matching objective. During imagination, the denoiser runs reverse diffusion from Gaussian noise to generate a predicted next-state  $\hat{z}_{t+1}$  in  $K$  denoising steps.

The diffusion formulation captures the multi-modal, stochastic nature of protein fitness landscapes: a single mutation can lead to distinct structural or functional outcomes depending on context. By sampling  $N$  denoising trajectories, the model produces a distributional estimate whose variance directly quantifies uncertainty. This built-in uncertainty feeds the Active Inference exploration mechanism without requiring external modules.

The total training loss combines: (i) the denoising loss for learning dynamics, (ii) SIGReg for preventing encoder collapse, and (iii) a supervised reward loss for grounding the latent space in experimental fitness measurements. The complete loss formulation and diffusion equations are given in Appendix A, and the training procedure is formalized in Algorithm 1.

## 4.5 Alternative: Energy-Based JEPA (Architecture B)

As a faster alternative, Architecture B replaces the diffusion predictor with a deterministic Transformer that maps  $(z_t, a_t^{\text{emb}})$  to  $\hat{z}_{t+1}$  in a single forward pass. The training loss consists of a prediction error against the EMA target latent, SIGReg, and the reward loss. Since the deterministic predictor yields no distributional samples, uncertainty is estimated externally via an ensemble of  $M$  predictors or an evidential deep learning head that outputs a Normal-Inverse-Gamma distribution. Architecture B is faster (no denoising iterations) and suitable for rapid screening, but averages over modes in the fitness landscape. The deterministic prediction loss and ensemble uncertainty formulation are detailed in Appendix A. The natural progression is to start with Architecture B for initial experiments and upgrade to Architecture A for final design campaigns.

## 4.6 Reward Head

The reward head  $R_\psi(z_t)$  is a multi-task MLP with a shared backbone and task-specific output heads predicting stability ( $\Delta\Delta G$ ), binding affinity ( $K_d$ ), and activity ( $k_{\text{cat}}$ ). It is trained supervised on DMS data from ProteinGym and Tsuboyama’s mega-scale stability measurements using a SmoothL1 loss (see Appendix A). Weights  $w_k$  balance the task-specific losses.

## 4.7 Active Inference Policy

The policy  $\pi_\xi(a_t|z_t)$  selects mutations by minimizing expected free energy  $G(\pi)$  [15], which decomposes into:

- **Pragmatic value:** the expected negative reward over the imagined trajectory, driving the agent toward high-fitness sequences.
- **Epistemic value:** the expected reduction in model uncertainty, driving the agent toward mutations about which the world model is most uncertain.

The implementation uses a latent-space actor-critic. The actor outputs a categorical distribution over mutations. The critic estimates the expected negative free energy from each state. Both are trained on imagined trajectories: every  $N_{\text{imagine}}$  steps, the policy rolls out  $H$  mutation steps in the world model, collecting predicted rewards and uncertainty estimates. The critic targets are computed via  $\text{TD}(\lambda_{\text{td}})$  returns on the imagined data (see Appendix A for the expected free energy decomposition and actor-critic update rules, and Algorithm 2 for the full imagination-based training loop). An alternative policy is Monte Carlo Tree Search over mutation trees, scored by the reward head, which is more sample-efficient for short planning horizons.

## 4.8 Training Procedure

Training proceeds in two alternating phases. In the *world model phase*, batches of  $(s_t, a_t, r_t, s_{t+1})$  transitions from DMS datasets are used to update the encoder, predictor, and reward head via the combined training loss. The EMA target encoder is updated after each step. In the *policy phase*, imagined rollouts from the trained world model are used to update the actor and critic. The two phases alternate, with world model updates occurring at every step and policy updates every  $N_{\text{imagine}}$  steps. Algorithms 1, 2, and 3 in Appendix B formalize the world model training, policy training, and inference procedures respectively.

## 4.9 Data

Training data comes from three main sources: (1) ProteinGym [21], providing 217 substitution DMS assays and 66 indel assays across diverse protein families, (2) the Tsuboyama mega-scale stability dataset [22], providing over 776,000  $\Delta G$  measurements, and (3) ESM-Fold/AlphaFold2 predicted structures for all sequences, serving as cheap oracles for the structure encoder. Evaluation will use held-out ProteinGym assays, and multi-step planning will be validated on known evolutionary paths (e.g., TEM-1  $\beta$ -lactamase resistance evolution).

# 5 Expected Contributions and Timeline

## 5.1 Scientific Contributions

1. **Latent Diffusion JEPA for proteins:** A new world model architecture that combines intervention-conditioned JEPA prediction [9, 10] with diffusion-based dynamics [37, 35] in latent space. This is the first application of JEPA world models to protein engineering and the first latent diffusion world model for biological sequences.
2. **JEPA–Active Inference unification:** A formal connection between the JEPA energy-based objective [8] and the variational free energy of Active Inference [15], providing a principled theoretical framework where the world model training objective and the policy selection criterion derive from the same variational principle.
3. **Multi-step mutation planning:** Empirical demonstration that world-model-based planning can navigate epistatic fitness landscapes, crossing fitness valleys that are inaccessible to greedy single-step methods.
4. **Open-source framework:** A modular library (**protein-dreamer**) where the encoder, predictor, reward head, and policy are independently swappable, supporting

both generative and non-generative world model backends.

## 5.2 Timeline

**Year 1: Foundations and Energy-Based JEPA.** Build Architecture B: ESM-2 + GVP-GNN encoder with SIGReg, deterministic JEPA predictor, fitness reward head, PPO policy in latent space. Benchmark on ProteinGym single-mutant landscapes. Target output: workshop paper or preprint on “JEPA World Models for Protein Fitness Landscapes.”

**Year 2: Latent Diffusion JEPA and Active Inference.** Replace the deterministic predictor with the conditional latent diffusion model [37, 35]. Integrate the Active Inference policy [15] with uncertainty-driven exploration. Validate multi-step planning on known evolutionary paths. Target output: top-venue publication (NeurIPS, ICML, or Nature Methods).

**Year 3: Multi-Objective Design and Active Learning.** Extend to multi-objective optimization (stability + binding + activity). Implement an active learning loop where the world model is fine-tuned on small batches of new experimental data, closing the compute-experiment loop. Target output: application paper demonstrating real-world protein engineering campaigns.

**Year 4: Thesis Writing and Extensions.** Consolidate results, release the `protein-dreamer` library, and explore extensions including meta-learning across fitness landscapes, integration with GFlowNet [30] policies for diversity-seeking design, and generalizing the action space beyond single-site substitutions to insertions, deletions, and multi-site edits. Supporting indels would require redesigning the action representation and adapting the world model to variable-length latent states, but would bring the framework closer to the full space of natural protein evolution.

## 6 Starting Project: JEPA World Models for Protein Fitness Landscapes

The first year of the PhD will produce a self-contained study that validates the core hypothesis: that a JEPA world model can learn useful representations of protein fitness landscapes, and that model-based planning in latent space outperforms standard fitness predictors for protein design.

The starting project focuses on Architecture B (Energy-Based JEPA), which is simpler and faster to train. The encoder combines ESM-2 [17] sequence embeddings with GVP-GNN [19] structural features, fused and regularized with SIGReg [10]. The predictor is

a deterministic Transformer mapping  $(z_t, a_t) \rightarrow \hat{z}_{t+1}$ . The reward head predicts stability from latent states, trained on Tsuboyama mega-scale data [22]. The policy is PPO with an ensemble-based uncertainty bonus.

**Experimental plan:**

1. **World model accuracy:** Evaluate whether the world model’s predicted latent transitions are consistent with actual fitness changes across single-mutation steps on ProteinGym [21] benchmarks. Compare the reward head’s fitness predictions against ESM-2 zero-shot, Tranception, and S3F [20] baselines.
2. **Multi-step rollout quality:** Starting from wild-type proteins, roll out sequences of 2–5 mutations in imagination and measure whether predicted fitness trajectories correlate with known experimental trajectories (e.g., from combinatorial DMS data with double/triple mutants).
3. **Design performance:** Use ProteinDreamer to propose mutation sets for 10–20 ProteinGym assays. Evaluate proposed sequences against the ground-truth fitness landscape using top-K recall and NDCG metrics. Compare with DynaPPO [12],  $\mu$ Search [13], and random mutation baselines.
4. **Ablation studies:** Measure the contribution of each component: structure encoder (GVP-GNN vs. sequence-only), SIGReg (vs. VICReg vs. no regularization), planning horizon (greedy vs. 3-step vs. 5-step), and Active Inference exploration (vs. entropy bonus vs. no exploration).

This project is expected to produce a workshop paper or preprint, providing the empirical groundwork for the subsequent Latent Diffusion JEPA development and future submissions to top machine learning or computational biology venues.

## References

- [1] Sandhu, M., Chen, J. Z., Matthews, D. S., Spence, M. A., Pulsford, S. B., Gall, B., ... & Jackson, C. J. (2025). Computational and experimental exploration of protein fitness landscapes: navigating smooth and rugged terrains. *Biochemistry*, 64(8), 1673-1684.
- [2] Dauparas, J., Anishchenko, I., Bennett, N., Bai, H., Ragotte, R. J., Milles, L. F., ... & Baker, D. (2022). Robust deep learning-based protein sequence design using ProteinMPNN. *Science*, 378(6615), 49-56.
- [3] Watson, J. L., Juergens, D., Bennett, N. R., Trippe, B. L., Yim, J., Eisenach, H. E., ... & Baker, D. (2023). De novo design of protein structure and function with RFdiffusion. *Nature*, 620(7976), 1089-1100.
- [4] Hayes, T., Rao, R., Akin, H., Sofroniew, N. J., Oktay, D., Lin, Z., ... & Wiggert, M. (2024, July). Simulating 500 million years of evolution with a language model. *bioRxiv*.
- [5] Notin, P., Rollins, N., Gal, Y., Sander, C., & Marks, D. (2024). Machine learning for functional protein design. *Nature biotechnology*, 42(2), 216-228.
- [6] Kyro, G. W., Qiu, T., & Batista, V. S. (2025). A model-centric review of deep learning for protein design. *arXiv preprint arXiv:2502.19173*.
- [7] Hafner, D., Yan, W., & Lillicrap, T. (2025). Training agents inside of scalable world models. *arXiv preprint arXiv:2509.24527*.
- [8] LeCun, Y. (2022). A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, 62(1), 1-62.
- [9] Nam, H., Lidec, Q. L., Maes, L., LeCun, Y., & Balestrieri, R. (2026). Causal-JEPA: Learning World Models through Object-Level Latent Interventions. *arXiv preprint arXiv:2602.11389*.
- [10] Maes, L., Lidec, Q. L., Scieur, D., LeCun, Y., & Balestrieri, R. (2026). LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels. *arXiv preprint arXiv:2603.19312*.
- [11] Alonso, E., Jelley, A., Micheli, V., Kanervisto, A., Storkey, A., Pearce, T., & Fleuret, F. (2024). Diffusion for world modeling: Visual details matter in atari. *Advances in Neural Information Processing Systems*, 37, 58757-58791.

- [12] Angermueller, C., Dohan, D., Belanger, D., Deshpande, R., Murphy, K., & Colwell, L. (2019). Model-based reinforcement learning for biological sequence design. *In International conference on learning representations*.
- [13] Sun, H., He, L., Deng, P., Liu, G., Zhao, Z., Jiang, Y., ... & Liu, T. Y. (2025). Accelerating protein engineering with fitness landscape modelling and reinforcement learning. *Nature Machine Intelligence*, 7(9), 1446-1460.
- [14] Wang, Y., Tang, H., Huang, L., Pan, L., Yang, L., Yang, H., ... & Yang, M. (2023). Self-play reinforcement learning guides protein engineering. *Nature Machine Intelligence*, 5(8), 845-860.
- [15] Parr, T., Pezzulo, G., & Friston, K. J. (2022). Active inference: the free energy principle in mind, brain, and behavior. *MIT Press*.
- [16] Tschantz, A., Millidge, B., Seth, A. K., & Buckley, C. L. (2020). Reinforcement learning through active inference. *arXiv preprint arXiv:2002.12636*.
- [17] Lin, Z., Akin, H., Rao, R., Hie, B., Zhu, Z., Lu, W., ... & Rives, A. (2023). Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637), 1123-1130.
- [18] Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., ... & Hassabis, D. (2021). Highly accurate protein structure prediction with AlphaFold. *nature*, 596(7873), 583-589.
- [19] Jing, B., Eismann, S., Suriana, P., Townshend, R. J., & Dror, R. (2020). Learning from protein structure with geometric vector perceptrons. *arXiv preprint arXiv:2009.01411*.
- [20] Zhang, Z., Notin, P., Huang, Y., Lozano, A., Chenthamarakshan, V., Marks, D., ... & Tang, J. (2024). Multi-scale representation learning for protein fitness prediction. *Advances in Neural Information Processing Systems*, 37, 101456-101473.
- [21] Notin, P., Kollasch, A., Ritter, D., Van Niekerk, L., Paul, S., Spinner, H., ... & Marks, D. (2023). Proteingym: Large-scale benchmarks for protein fitness prediction and design. *Advances in neural information processing systems*, 36, 64331-64379.
- [22] Tsuboyama, K., Dauparas, J., Chen, J., Laine, E., Mohseni Behbahani, Y., Weinstein, J. J., ... & Rocklin, G. J. (2023). Mega-scale experimental analysis of protein folding stability in biology and design. *Nature*, 620(7973), 434-444.
- [23] Chen, L., Zhang, Z., Li, Z., Li, R., Huo, R., Chen, L., ... & Zheng, M. (2023). Learning protein fitness landscapes with deep mutational scanning data from multiple sources. *Cell Systems*, 14(8), 706-721.

- [24] Alamdari, S., Thakkar, N., Van Den Berg, R., Tenenholtz, N., Strome, R., Moses, A. M., ... & Yang, K. K. (2023). Protein generation with evolutionary diffusion: sequence is all you need. *BioRxiv*, 2023-09.
- [25] Nijkamp, E., Ruffolo, J. A., Weinstein, E. N., Naik, N., & Madani, A. (2023). Progen2: exploring the boundaries of protein language models. *Cell systems*, 14(11), 968-978.
- [26] Lee, M., Vecchietti, L. F., Jung, H., Ro, H., Cha, M., & Kim, H. M. (2023). Protein sequence design in a latent space via model-based reinforcement learning.
- [27] Renard, F., Courtot, C., Reichlin, A., & Bent, O. (2024). Model-based reinforcement learning for protein backbone design. *arXiv preprint arXiv:2405.01983*.
- [28] Sternke, M., & Karpiak, J. (2023). ProteinRL: Reinforcement learning with generative protein language models for property-directed sequence design. *In NeurIPS 2023 Generative AI and Biology (GenBio) Workshop*.
- [29] Subramanian, J., Sujit, S., Irtisam, N., Sain, U., Islam, R., Nowrouzezahrai, D., & Kahou, S. E. (2024). Reinforcement learning for sequence design leveraging protein language models. *arXiv preprint arXiv:2407.03154*.
- [30] Jain, M., Bengio, E., Hernandez-Garcia, A., Rector-Brooks, J., Dossou, B. F., Ekbote, C. A., ... & Bengio, Y. (2022, June). Biological sequence design with gflownets. *In International conference on machine learning (pp. 9786-9801)*. PMLR.
- [31] Hie, B. L., & Yang, K. K. (2022). Adaptive machine learning for protein engineering. *Current opinion in structural biology*, 72, 145-152.
- [32] Ding, K., Chin, M., Zhao, Y., Huang, W., Mai, B. K., Wang, H., ... & Luo, Y. (2024). Machine learning-guided co-optimization of fitness and diversity facilitates combinatorial library design in enzyme engineering. *Nature Communications*, 15(1), 6392.
- [33] Sutton, R. S. (1991). Dyna, an integrated architecture for learning, planning, and reacting. *ACM Sigart Bulletin*, 2(4), 160-163.
- [34] Ha, D., & Schmidhuber, J. (2018). World models. *arXiv preprint arXiv:1803.10122*, 2(3), 440.
- [35] Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. *In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 10684-10695).
- [36] Sajid, N., Ball, P. J., Parr, T., & Friston, K. J. (2021). Active inference: demystified and compared. *Neural Computation*, 33(3), 674-712.

- [37] Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33, 6840-6851.

# A Mathematical Details

## A.1 Notation

Table 1 summarizes the principal symbols and notation used throughout this proposal.

Table 1: Summary of notation.

| Symbol                                             | Meaning                                           |
|----------------------------------------------------|---------------------------------------------------|
| $s_t = (\mathbf{x}_t, \mathbf{C}_t, \mathbf{p}_t)$ | Protein state (sequence, coordinates, properties) |
| $a_t$                                              | Mutation action (position, new amino acid)        |
| $z_t \in \mathbb{R}^d$                             | Latent representation of protein state            |
| $\bar{z}_{t+1}$                                    | Target encoder latent (EMA, stop-gradient)        |
| $a_t^{\text{emb}} \in \mathbb{R}^{d_a}$            | Continuous mutation embedding                     |
| $f_\theta$                                         | Context encoder                                   |
| $f_{\bar{\theta}}$                                 | Target encoder (EMA of $f_\theta$ )               |
| $e_\alpha$                                         | Action encoder                                    |
| $D_{\hat{\theta}}$                                 | Diffusion denoiser (Architecture A)               |
| $g_\phi$                                           | Deterministic predictor (Architecture B)          |
| $R_\psi$                                           | Reward head                                       |
| $\pi_\xi$                                          | Policy (actor)                                    |
| $V_\omega$                                         | Value function (critic)                           |
| $\tau_{\text{ema}}$                                | EMA decay rate                                    |
| $\tau$                                             | Diffusion timestep index ( $1 \leq \tau \leq K$ ) |
| $\lambda_{\text{reg}}$                             | SIGReg regularization weight                      |
| $\lambda_{\text{td}}$                              | Bootstrapping parameter for TD returns            |
| $K$                                                | Number of diffusion steps                         |
| $N$                                                | Number of diffusion samples for uncertainty       |
| $M$                                                | Number of ensemble members (Architecture B)       |
| $H$                                                | Planning horizon (imagination rollout length)     |
| $\mathcal{L}_{\text{diff}}$                        | Denoising score-matching objective                |
| $\mathcal{L}_{\text{pred}}$                        | JEPA prediction loss                              |
| $\mathcal{L}_{\text{rew}}$                         | Supervised fitness prediction loss                |
| $\text{SIGReg}(z_t)$                               | Sketched-Isotropic-Gaussian regularizer           |
| $G(\pi)$                                           | Expected free energy of policy $\pi$              |
| $\bar{\alpha}_\tau$                                | Cumulative noise schedule product                 |

## A.2 MDP Formulation

The protein design MDP is  $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$  with:

$$s_t = (\mathbf{x}_t, \mathbf{C}_t, \mathbf{p}_t), \quad \mathbf{x}_t \in \{A, C, D, \dots, Y\}^L, \quad \mathbf{C}_t \in \mathbb{R}^{L \times 3} \quad (1)$$

$$a_t \in \mathcal{A} = \{(i, j) : i \in \{1, \dots, L\}, j \in \{1, \dots, 20\}\} \quad (2)$$

$$z_t = f_\theta(s_t), \quad \bar{z}_{t+1} = f_{\bar{\theta}}(s_{t+1}) \quad (3)$$

where  $\mathbf{p}_t = (\text{pLDDT}_t, \mathbf{M}_t^{\text{contact}})$  collects per-residue confidence and predicted contact maps from ESMFold.

### A.3 Context Encoder

$$\mathbf{H}^{\text{seq}} = \text{ESM-2}(\mathbf{x}_t) \in \mathbb{R}^{L \times 1280}, \quad \mathbf{h}^{\text{seq}} = \text{MeanPool}(\mathbf{H}^{\text{seq}}) \in \mathbb{R}^{1280} \quad (4)$$

$$\mathbf{H}^{\text{struct}} = \text{GVP-GNN}(\mathbf{C}_t) \in \mathbb{R}^{L \times 1280}, \quad \mathbf{h}^{\text{struct}} = \text{MeanPool}(\mathbf{H}^{\text{struct}}) \in \mathbb{R}^{1280} \quad (5)$$

$$z_t = \text{FusionMLP}([\mathbf{h}^{\text{seq}} \parallel \mathbf{h}^{\text{struct}}]) \in \mathbb{R}^d \quad (6)$$

The GVP-GNN message passing operates on a protein graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$  where edges connect residues within a distance cutoff:

$$\mathbf{m}_{ij} = \text{GVP}\left([\mathbf{s}_i \parallel \mathbf{s}_j \parallel \|\vec{r}_{ij}\|], [\vec{v}_i \parallel \vec{v}_j \parallel \vec{r}_{ij}]\right) \quad (7)$$

$$\mathbf{s}'_i, \vec{v}'_i = \text{GVP}\left(\mathbf{s}_i + \sum_{j \in \mathcal{N}(i)} \mathbf{m}_{ij}^{(s)}, \vec{v}_i + \sum_{j \in \mathcal{N}(i)} \mathbf{m}_{ij}^{(v)}\right) \quad (8)$$

### A.4 Fusion MLP

The sequence and structure representations are fused into a single latent state:

$$z_t = \text{FusionMLP}([\mathbf{h}^{\text{seq}} \parallel \mathbf{h}^{\text{struct}}]) = \text{LayerNorm}\left(\text{MLP}([\mathbf{h}^{\text{seq}} \parallel \mathbf{h}^{\text{struct}}])\right) \in \mathbb{R}^d \quad (9)$$

### A.5 Action Encoder

$$a_t^{\text{emb}} = e_\alpha(a_t) = \text{MLP}\left([\text{PosEmb}(i) \parallel \text{AAEmb}(\text{AA}_{\text{old}}) \parallel \text{AAEmb}(\text{AA}_{\text{new}})]\right) \in \mathbb{R}^{d_a} \quad (10)$$

### A.6 Target Encoder (EMA)

$$\bar{\theta} \leftarrow \tau_{\text{ema}} \bar{\theta} + (1 - \tau_{\text{ema}}) \theta, \quad \bar{z}_{t+1} = f_{\bar{\theta}}(s_{t+1}) \quad (\text{stop-gradient}) \quad (11)$$

### A.7 Architecture A: Latent Diffusion JEPA

The diffusion dynamics follow the Denoising Diffusion Probabilistic Model (DDPM) formulation of Ho et al. [37], adapted to operate in the JEPA latent space rather than observation space, inspired by the Latent Diffusion Model (LDM) framework of Rombach et al. [35].

**Forward process** (cosine noise schedule,  $K$  steps):

$$\bar{z}_{t+1}^\tau = \sqrt{\bar{\alpha}_\tau} \bar{z}_{t+1} + \sqrt{1 - \bar{\alpha}_\tau} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I}), \quad \bar{\alpha}_\tau = \prod_{i=1}^\tau \alpha_i \quad (12)$$

**Denoiser** ( $x_0$ -prediction):

$$\hat{z}_{t+1}^0 = D_{\hat{\theta}}(\bar{z}_{t+1}^\tau, \tau, z_t, a_t^{\text{emb}}) \quad (13)$$

**Reverse process** (during imagination,  $x_0$ -prediction form):

$$\hat{z}_{t+1}^{\tau-1} = \frac{\sqrt{\bar{\alpha}_{\tau-1}} \beta_\tau}{1 - \bar{\alpha}_\tau} D_{\hat{\theta}}(\hat{z}_{t+1}^\tau, \tau, z_t, a_t^{\text{emb}}) + \frac{\sqrt{\alpha_\tau} (1 - \bar{\alpha}_{\tau-1})}{1 - \bar{\alpha}_\tau} \hat{z}_{t+1}^\tau + \sigma_\tau \mathbf{w}, \quad \mathbf{w} \sim \mathcal{N}(0, \mathbf{I}) \quad (14)$$

where  $\beta_\tau = 1 - \alpha_\tau$  and  $\sigma_\tau^2 = \tilde{\beta}_\tau = \frac{(1 - \bar{\alpha}_{\tau-1}) \beta_\tau}{1 - \bar{\alpha}_\tau}$ .

**Training loss:**

$$\boxed{\mathcal{L}_A = \beta_{\text{diff}} \cdot \mathcal{L}_{\text{diff}} + \lambda_{\text{reg}} \cdot \text{SIGReg}(z_t) + \beta_{\text{rew}} \cdot \mathcal{L}_{\text{rew}}} \quad (15)$$

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{\tau \sim \mathcal{U}(1, K), \epsilon \sim \mathcal{N}(0, \mathbf{I})} \left[ \left\| D_{\hat{\theta}}(\bar{z}_{t+1}^\tau, \tau, z_t, a_t^{\text{emb}}) - \bar{z}_{t+1} \right\|_2^2 \right] \quad (16)$$

Default loss weights:  $\beta_{\text{diff}}$ ,  $\lambda_{\text{reg}}$ , and  $\beta_{\text{rew}}$  are tuned as hyperparameters.

## A.8 Architecture B: Energy-Based JEPA

**Deterministic predictor:**

$$\hat{z}_{t+1} = g_\phi(z_t, a_t^{\text{emb}}) = \text{TransformerBlock}([z_t \| a_t^{\text{emb}}])[:, d] \quad (17)$$

**Training loss:**

$$\boxed{\mathcal{L}_B = \beta_{\text{jepa}} \cdot \mathcal{L}_{\text{pred}} + \lambda_{\text{reg}} \cdot \text{SIGReg}(z_t) + \beta_{\text{rew}} \cdot \mathcal{L}_{\text{rew}}} \quad (18)$$

$$\mathcal{L}_{\text{pred}} = \frac{1}{B} \sum_{i=1}^B \left\| g_\phi(z_t^{(i)}, a_t^{(i)\text{emb}}) - \text{sg}(\bar{z}_{t+1}^{(i)}) \right\|_2^2 \quad (19)$$

where  $\text{sg}(\cdot)$  denotes the stop-gradient operator.

## A.9 Reward Head

$$\hat{r}_t = R_\psi(z_t) = [\hat{y}_t^{\text{stab}}, \hat{y}_t^{\text{bind}}, \hat{y}_t^{\text{act}}], \quad \mathcal{L}_{\text{rew}} = \sum_k w_k \cdot \text{SmoothL1}(\hat{y}_t^{(k)}, y_t^{(k)}) \quad (20)$$

## A.10 Uncertainty Quantification

**Architecture A** (built-in, from  $N$  diffusion samples):

$$\text{Unc}(z_t, a_t) = \frac{1}{N} \sum_{n=1}^N \left\| \hat{z}_{t+1}^{(n)} - \bar{z}_{t+1} \right\|_2^2, \quad \bar{z}_{t+1} = \frac{1}{N} \sum_{n=1}^N \hat{z}_{t+1}^{(n)} \quad (21)$$

**Architecture B** (ensemble of  $M$  predictors):

$$\text{Unc}_{\text{ens}}(z_t, a_t) = \frac{1}{M} \sum_{m=1}^M \left\| g_{\phi_m}(z_t, a_t^{\text{emb}}) - \bar{g}(z_t, a_t^{\text{emb}}) \right\|_2^2 \quad (22)$$

where  $\bar{g}(z_t, a_t^{\text{emb}}) = \frac{1}{M} \sum_{m=1}^M g_{\phi_m}(z_t, a_t^{\text{emb}})$  is the ensemble mean predictor.

## A.11 Active Inference Policy

**Expected Free Energy:**

$$G(\pi) = \underbrace{\mathbb{E}_{q_\phi} \left[ \sum_{t=1}^H -r_t \right]}_{\text{Pragmatic value}} + \underbrace{\mathbb{E}_{q_\phi} \left[ \sum_{t=1}^H -\eta \cdot \text{Unc}(z_t, a_t) \right]}_{\text{Epistemic value}}, \quad \pi^* = \arg \min_{\pi} G(\pi) \quad (23)$$

**Actor update** (policy gradient):

$$\nabla_{\xi} J = \mathbb{E} \left[ \nabla_{\xi} \log \pi_{\xi}(a_t | z_t) \cdot (-G_t) \right] \quad (24)$$

**Critic update** (TD( $\lambda_{\text{td}}$ ) on imagined trajectories):

$$\mathcal{L}_{\text{critic}} = \mathbb{E} \left[ (V_{\omega}(z_t) - V_t^{\lambda_{\text{td}}})^2 \right], \quad V_t^{\lambda_{\text{td}}} = (1 - \lambda_{\text{td}}) \sum_{n=1}^{H-t} \lambda_{\text{td}}^{n-1} V_t^{(n)} \quad (25)$$

where  $V_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k g_{t+k} + \gamma^n V_{\omega}(z_{t+n})$  is the  $n$ -step bootstrapped return.

## B Training and Inference Algorithms

---

**Algorithm 1** ProteinDreamer – World Model Training (Architecture A)

---

**Require:** Replay buffer  $\mathcal{D}$  of  $(s_t, a_t, r_t, s_{t+1})$  from DMS data, diffusion steps  $K$

**Ensure:** Trained encoder  $f_{\theta}$ , denoiser  $D_{\hat{\theta}}$ , action encoder  $e_{\alpha}$ , reward head  $R_{\psi}$

- 1: **repeat**
  - 2:   Sample batch  $\{(s_t, a_t, r_t, s_{t+1})\}$  from  $\mathcal{D}$
  - 3:    $z_t \leftarrow f_{\theta}(s_t), \quad \bar{z}_{t+1} \leftarrow f_{\bar{\theta}}(s_{t+1})$    [stop-gradient on  $\bar{z}_{t+1}$ ]
  - 4:    $a_t^{\text{emb}} \leftarrow e_{\alpha}(a_t)$
  - 5:   Sample  $\tau \sim \mathcal{U}(1, K), \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I})$
  - 6:    $\bar{z}_{t+1}^{\tau} \leftarrow \sqrt{\bar{\alpha}_{\tau}} \cdot \bar{z}_{t+1} + \sqrt{1 - \bar{\alpha}_{\tau}} \cdot \varepsilon$
  - 7:    $\hat{z}_{t+1}^0 \leftarrow D_{\hat{\theta}}(\bar{z}_{t+1}^{\tau}, \tau, z_t, a_t^{\text{emb}})$
  - 8:    $\mathcal{L}_A \leftarrow \beta_{\text{diff}} \|\hat{z}_{t+1}^0 - \bar{z}_{t+1}\|^2 + \lambda_{\text{reg}} \cdot \text{SIGReg}(z_t) + \beta_{\text{rew}} \cdot \mathcal{L}_{\text{rew}}$
  - 9:   Update  $\theta, \bar{\theta}, \alpha, \psi$  via  $\nabla \mathcal{L}_A$
  - 10:    $\bar{\theta} \leftarrow \tau_{\text{ema}} \cdot \bar{\theta} + (1 - \tau_{\text{ema}}) \cdot \bar{\theta}$    [EMA update]
  - 11: **until** convergence
-

---

**Algorithm 2** ProteinDreamer – Imagination-Based Policy Training

---

**Require:** Trained world model  $(f_\theta, D_{\hat{\theta}}, R_\psi)$ , planning horizon  $H$ , diffusion samples  $N$

**Ensure:** Trained actor  $\pi_\xi$ , critic  $V_\omega$

```
1: repeat
2:   Sample initial  $z_0$  from encoded replay buffer states
3:   for  $t = 0$  to  $H - 1$  do
4:      $a_t \sim \pi_\xi(\cdot|z_t)$  [sample mutation from policy]
5:     Sample  $N$  latents:  $\{\hat{z}_{t+1}^{(n)}\}_{n=1}^N \sim p_\theta(\cdot|z_t, a_t)$  [ $N$  diffusion rollouts]
6:      $\hat{z}_{t+1} \leftarrow \text{mean}(\{\hat{z}_{t+1}^{(n)}\})$  [mean for forward pass]
7:      $\hat{r}_t \leftarrow R_\psi(\hat{z}_{t+1})$  [predict fitness]
8:      $u_t \leftarrow \text{Var}(\{\hat{z}_{t+1}^{(n)}\})$  [uncertainty from sample variance]
9:      $g_t \leftarrow -\hat{r}_t - \eta \cdot u_t$  [expected free energy]
10:     $z_{t+1} \leftarrow \hat{z}_{t+1}$ 
11:  end for
12:  Compute  $\lambda_{\text{td}}$ -returns  $V_t^{\lambda_{\text{td}}}$  from  $\{g_t, \dots, g_{H-1}\}$ 
13:  Update critic:  $\nabla_\omega \|V_\omega(z_t) - V_t^{\lambda_{\text{td}}}\|^2$ 
14:  Update actor:  $\nabla_\xi \mathbb{E}[\log \pi_\xi(a_t|z_t) \cdot (-g_t - V_\omega(z_t))]$ 
15: until convergence
```

---

---

**Algorithm 3** ProteinDreamer – Inference (Protein Design)

---

**Require:** Wild-type protein  $s_0$ , trained world model and policy, design budget  $T$ , candidates  $C$

**Ensure:** Ranked set of designed protein sequences

```
1:  $z_0 \leftarrow f_\theta(s_0)$  [encode wild-type]
2: Initialize candidate set  $\mathcal{C} \leftarrow \emptyset$ 
3: for  $c = 1$  to  $C$  do
4:    $z \leftarrow z_0$ 
5:   for  $t = 0$  to  $T - 1$  do
6:      $a_t \sim \pi_\xi(\cdot|z)$  [sample mutation]
7:      $\hat{z} \leftarrow \text{mean}(\{\hat{z}^{(n)}\}_{n=1}^N \sim p_\theta(\cdot|z, a_t))$ 
8:      $z \leftarrow \hat{z}$ 
9:   end for
10:   $\hat{r} \leftarrow R_\psi(z)$  [score final state]
11:  Apply mutations  $\{a_0, \dots, a_{T-1}\}$  to wild-type sequence
12:   $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\text{designed\_seq}, \hat{r})\}$ 
13: end for
14: return  $\mathcal{C}$  ranked by  $\hat{r}$  [optionally validate top-k with ESMFold/AlphaFold2]
```

---